

PRACTICA INTEGRADORA DE PROGRAMACION ORIENTADA A OBJETOS

CRUD CON HERENCIA, ARRAYLIST Y VALIDACIONES EN JAVA

Asignatura	Programacion Orientada a Objetos
Tema	Herencia, ArrayList, CRUD y manejo de excepciones
Modalidad	Trabajo practico en Java consola
Producto esperado	Sistema funcional, UML, capturas, README y repositorio GitHub

Objetivo de aprendizaje: Aplicar Programacion Orientada a Objetos en Java mediante una aplicacion de consola que integre herencia, encapsulamiento, sobrescritura, ArrayList, CRUD, menu interactivo y validacion de errores con try-catch.

1. Contexto del problema

Una institucion educativa necesita un sistema basico para registrar personas pertenecientes a la comunidad academica. Existen dos tipos de personas: estudiantes y docentes. Ambos comparten informacion general, pero cada uno posee datos especificos que deben gestionarse correctamente.

Clase padre	Subclase 1	Subclase 2
Persona - Cedula - Nombre completo - Edad	Estudiante Hereda de Persona - Carrera	Docente Hereda de Persona - Asignatura

2. Requerimiento técnico

Desarrolle una aplicacion en Java consola aplicando Programacion Orientada a Objetos. El sistema debe permitir administrar registros de estudiantes y docentes mediante operaciones CRUD.

2.1 Herencia

- Crear una clase padre llamada Persona con cedula, nombre completo y edad.
- Crear la subclase Estudiante, que hereda de Persona y agrega el atributo carrera.
- Crear la subclase Docente, que hereda de Persona y agrega el atributo asignatura.
- Sobrescribir el metodo para mostrar datos en cada subclase usando @Override.

2.2 CRUD con ArrayList

Operacion	Accion requerida	Metodo sugerido
CREATE	Registrar nuevos estudiantes o docentes.	add()
READ	Mostrar todos los registros almacenados.	get() / recorrido con for
UPDATE	Actualizar informacion de un registro existente.	set() o reemplazo controlado
DELETE	Eliminar un registro seleccionado.	remove()

2.3 Menu interactivo obligatorio

1. Registrar persona

2. Mostrar registros

3. Actualizar registro

4. Eliminar registro

5. Salir

El menu debe repetirse hasta que el usuario seleccione la opcion Salir.

3. Validaciones obligatorias

Situacion	Ejemplo de error	Mensaje esperado
Opcion de menu incorrecta	8, hola, -1	Error: opcion invalida. Intente nuevamente.
Ingreso incorrecto de numeros	veinte, abc	Error: debe ingresar solo numeros.
Posicion inexistente	99	Registro no encontrado.
Campos vacios	nombre vacio, cedula vacia, carrera vacia	Campo obligatorio.

Importante: el sistema no debe cerrarse por errores del usuario. Debe mostrar el mensaje correspondiente y permitir continuar con la ejecucion.

4. Restricciones de la practica

- No usar arreglos fijos como String[].
- No cerrar abruptamente el programa por errores de entrada.
- No trabajar con datos quemados como unica fuente de prueba.
- No entregar un codigo sin menu o sin validaciones.

5. Buenas practicas esperadas

- Aplicar encapsulamiento con atributos privados o protegidos segun corresponda.
- Usar constructores para inicializar objetos.
- Usar metodos separados para crear, mostrar, actualizar, eliminar y validar.
- Usar nombres claros para clases, atributos, metodos y variables.
- Usar mensajes comprensibles para orientar al usuario.
- Agregar comentarios breves en secciones importantes del codigo.

6. Ejemplo de ejecucion esperada

1. Registrar persona

Seleccione tipo:

1. Estudiante
2. Docente

Ingrese cedula: 1723456789

Ingrese nombre: Ana Lopez

Ingrese edad: 20

Ingrese carrera: Software

Registro agregado correctamente.

7. Plus reto adicional

Para obtener un punto adicional o reconocimiento de excelencia, el estudiante puede implementar una de las siguientes mejoras:

- Buscar una persona por cedula.
- Evitar que se registren dos personas con la misma cedula.
- Mostrar cuantas personas existen registradas por tipo: estudiantes y docentes.
- Agregar confirmacion antes de eliminar un registro.
- Separar las clases en archivos diferentes dentro de un paquete.

DEBE EL ESTUDIANTE DEFENDER EXPLICANDO EN CLASE, con buen conocimiento.

8. Entregables obligatorios

- Código fuente completo en Java.
- Diagrama UML de clases.
- Capturas de ejecución del sistema funcionando.
- Repositorio GitHub con el proyecto.
- Archivo README con explicación de conceptos aplicados.

9. Rubrica de evaluacion

Criterio	Excelente	Basico	Puntaje
Herencia y estructura de clases	Clase padre e hijas correctamente relacionadas.	Clases incompletas o relación poco clara.	2 pts
CRUD con ArrayList	Crear, leer, actualizar y eliminar funcionan correctamente.	Falta una operación o tiene errores.	3 pts
Validaciones y excepciones	Controla letras, opciones inválidas, campos vacíos y posiciones inexistentes.	Valida parcialmente o se cierra el programa.	3 pts
Menu interactivo	Menu claro, repetitivo y funcional.	Menu incompleto o poco claro.	1 pt
Buenas practicas POO	Código ordenado, modular y comprensible.	Código desordenado o poco comentado.	1 pt

10. Checklist antes de entregar

- ☐ El programa compila sin errores.
- ☐ El menu no se cierra si ingreso letras.
- ☐ El CRUD funciona con estudiantes y docentes.
- ☐ El ArrayList almacena objetos de tipo Persona.
- ☐ El UML coincide con el código entregado.
- ☐ El README explica herencia, ArrayList, CRUD y try-catch.
- ☐ Las capturas evidencian las pruebas realizadas.